

PORTADA

Técnicas de Programación

Unidad 7 — Desarrollo Web: CSS3: Fundamentos, Selectores y Modelo de Caja

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



TRANSICIÓN

Ya sabemos estructurar; hoy aprendemos a estilizar

En las dos sesiones anteriores construimos el **esqueleto** de una página web: estructura básica, texto, listas, enlaces e imágenes, formularios, tablas y etiquetas semánticas (`<header>` , `<nav>` , `<main>` , `<section>` , `<article>` , `<footer>`). El resultado es correcto, accesible y navegable — pero visualmente es texto plano en blanco y negro, sin colores, tipografía cuidada ni espaciado.

Hoy resolvemos eso: **CSS3**, el lenguaje que le da apariencia visual a esa estructura. Aprenderemos a seleccionar elementos, aplicarles estilo y entender el **modelo de caja** que rige el tamaño y espaciado de cada elemento en la página.

CONCEPTO

Qué es CSS3

CSS (Cascading Style Sheets, "hojas de estilo en cascada") es el lenguaje que describe **cómo se ve** un documento HTML: colores, tipografía, tamaños, espaciado y posición de cada elemento. **CSS3** es la revisión más reciente y ampliamente adoptada del lenguaje, dividida en módulos independientes (selectores, colores, cajas, flexbox, grid, animaciones...) que evolucionan por separado.

La idea central es la **separación de responsabilidades**: el HTML define *qué es* cada cosa (un título, una lista, un párrafo — su contenido y estructura), y el CSS define *cómo se ve* (color, tamaño, posición). Esta separación permite cambiar por completo la apariencia de un sitio sin tocar una sola línea de HTML, y reutilizar el mismo CSS en muchas páginas.

Cascada: cuando varias reglas CSS podrían aplicar al mismo elemento, el navegador decide cuál "gana" siguiendo un orden de prioridad — volveremos a esto al hablar de especificidad.

CONCEPTO

Tres formas de vincular CSS a un HTML

Forma	Cómo se escribe	Alcance
Inline	Atributo <code>style=""</code> directo en la etiqueta	Solo ese elemento
Interno	Etiqueta <code><style></code> dentro del <code><head></code>	Toda esa página
Externo	<code><link rel="stylesheet" href="estilos.css"></code> en el <code><head></code>	Todas las páginas que lo enlacen

En la práctica se usa casi siempre **CSS externo**, por dos razones concretas:

- **Mantenibilidad:** un solo archivo `.css` puede estilizar decenas de páginas — cambiar un color ahí lo cambia en todo el sitio, sin tocar cada HTML.
- **Cacheo del navegador:** el navegador descarga el archivo CSS una vez y lo reutiliza en cada página que lo enlaza, en vez de volver a descargarlo — el sitio carga más rápido.

El CSS inline solo se justifica para casos muy puntuales (un estilo generado dinámicamente por JavaScript, por ejemplo) — no como práctica general.

CONCEPTO

Sintaxis de una regla CSS

Una hoja de estilos es una lista de **reglas**. Cada regla tiene un **selector** (qué elemento(s) del HTML se van a estilizar) y un bloque de **declaraciones** entre llaves, cada una con una propiedad y un valor separados por dos puntos y terminada en punto y coma.

```
selector {  
  propiedad: valor;  
  propiedad: valor;  
}  
  
/* ejemplo real */  
p {  
  color: #16453f;  
  font-size: 18px;  
}
```

Aquí `p` es el selector (todos los párrafos de la página), y `color` / `font-size` son propiedades con sus valores. El siguiente bloque de la sesión es justamente conocer los distintos tipos de **selectores** disponibles.

CONCEPTO

Selectores: elemento, clase e id

Tipo	Sintaxis	Selecciona
Elemento	p { }	Todos los <p> de la página
Clase	.aviso { }	Todo elemento con class="aviso" (se puede repetir en varios elementos)
Id	#menu-principal { }	El único elemento con id="menu-principal" (el id no se repite en la página)

```
<p class="aviso">Cuidado</p>
<nav id="menu-principal">...</nav>

.aviso { color: #541734; font-weight: bold; }
#menu-principal { background: #16453f; }
```

Las clases son el selector **más usado en la práctica**: un mismo estilo se reutiliza en muchos elementos distintos de la página.

CONCEPTO

Selector descendiente y pseudo-clases

Un **selector descendiente** combina dos selectores separados por un espacio: selecciona un elemento solo cuando está **dentro de** otro, sin importar cuántos niveles de anidamiento haya.

```
div p { color: gray; }  
/* solo los <p> que están dentro de un <div>,  
   no todos los <p> de la página */
```

Las **pseudo-clases** seleccionan un elemento según un **estado** o **posición**, no según su etiqueta o atributo:

- `:hover` — mientras el mouse está sobre el elemento.
- `:first-child` — cuando el elemento es el primer hijo de su contenedor.

```
a:hover { color: #59a87d; }  
li:first-child { font-weight: bold; }
```

CONCEPTO

Especificidad: cuando dos reglas compiten

Si dos reglas CSS distintas aplican al **mismo elemento** con la misma propiedad, el navegador necesita decidir cuál gana. La regla práctica, de menor a mayor prioridad:

elemento < clase < id

Un selector de **id** siempre le gana a uno de **clase**, y un selector de **clase** siempre le gana a uno de **elemento** — sin importar el orden en que aparezcan escritas las reglas en el archivo.

```
p      { color: black; } /* elemento: pierde */
.avisos { color: green; } /* clase: pierde frente al id */
#titulo { color: red;   } /* id: gana */

<p id="titulo" class="avisos">¿De qué color queda?</p>
<!-- Rojo: el id tiene mayor especificidad -->
```

No es necesario memorizar la fórmula numérica completa de especificidad para empezar — con esta jerarquía básica (id > clase > elemento) es suficiente por ahora.

CONCEPTO

Colores en CSS

Forma	Ejemplo	Notas
Nombre	color: tomato;	Legible, pero solo ~150 nombres predefinidos
Hexadecimal	color: #59a87d;	El más usado; 6 dígitos hex (rojo-verde-azul)
rgb()	color: rgb(89, 168, 125);	Mismos tres canales, en base decimal 0-255

`color` pinta el **texto**; `background-color` pinta el **fondo** del elemento. Los tres formatos anteriores representan exactamente el mismo color de tres maneras distintas — se elige el que sea más conveniente según el contexto (un diseñador suele entregar valores hex).

CONCEPTO

Unidades: absolutas vs. relativas

Unidad	Tipo	Significa
px	Absoluta	Píxeles fijos — no cambia según el contexto
%	Relativa	Porcentaje del tamaño del elemento contenedor
em	Relativa	Múltiplo del tamaño de fuente del elemento actual (o su padre)
rem	Relativa	Múltiplo del tamaño de fuente raíz (<code><html></code>), sin importar el anidamiento

Las unidades **relativas** se recalculan según su referencia, lo que las hace más flexibles para diseño adaptable: si el usuario aumenta el tamaño de fuente del navegador, un texto en `rem` crece con él; uno en `px` se queda fijo.

```
html { font-size: 16px; }      /* referencia raíz */
h1   { font-size: 2rem;  }     /* 32px, siempre relativo a la raíz */
.card p { font-size: 1.2em; } /* 1.2× el tamaño de fuente del .card */
```

CONCEPTO

Tipografía básica

Propiedad	Controla
font-family	La fuente tipográfica (lista de opciones, con una genérica al final)
font-size	El tamaño del texto
font-weight	El grosor: <code>normal</code> , <code>bold</code> , o un número (100–900)
line-height	La altura de línea — espacio vertical entre renglones
text-align	Alineación horizontal: <code>left</code> , <code>center</code> , <code>right</code> , <code>justify</code>

```
p {
  font-family: "Segoe UI", Arial, sans-serif;
  font-size: 1rem;
  font-weight: normal;
  line-height: 1.5;
  text-align: left;
}
```

CONCEPTO

El modelo de caja (box model)

Todo elemento HTML es, para CSS, una **caja rectangular** formada por cuatro capas concéntricas. De adentro hacia afuera:

1. **Content** — el contenido real (texto, imagen), con ancho/alto definidos por `width / height`.
2. **Padding** — espacio interno entre el contenido y el borde; "acolchado" de la caja.
3. **Border** — el borde visible de la caja (grosor, estilo, color).
4. **Margin** — espacio externo entre esta caja y las cajas vecinas; no tiene color ni fondo.

```
.caja {  
  width: 200px;  
  padding: 20px;  
  border: 2px solid #16453f;  
  margin: 10px;  
}
```

Entender esta capa por capa es clave para el siguiente punto: **cuánto espacio ocupa realmente** un elemento con `width: 200px` en pantalla.

CONTRAEJEMPLO

La sorpresa del ancho: 200px que no miden 200px

Por defecto, el navegador calcula `width` aplicándolo **solo al content** — el padding y el border se **suman aparte**. Este comportamiento por defecto sorprende a quien empieza con CSS:

```
.caja {  
  width: 200px;  
  padding: 20px; /* 20px a cada lado */  
  border: 1px solid black; /* 1px a cada lado */  
}  
/* Ancho real en pantalla:  
   200 (content) + 20+20 (padding) + 1+1 (border) = 242px */
```

El elemento termina midiendo **242px**, no los 200px que el desarrollador escribió — y si varias cajas de "200px" se ponen una junto a otra esperando que sumen exactamente el ancho del contenedor, el layout se desborda.



CONCEPTO

box-sizing: border-box resuelve la sorpresa

La propiedad `box-sizing` cambia **qué incluye** el `width` declarado. Con `border-box`, el ancho que escribes ya incluye padding y border — el navegador ajusta el content hacia adentro para que el total **sí** mida lo declarado.

```
.caja {  
  box-sizing: border-box;  
  width: 200px;  
  padding: 20px;  
  border: 1px solid black;  
}  
/* Ancho real en pantalla: exactamente 200px.  
   El content se reduce a 200 - 40 - 2 = 158px */
```

Por esto es muy común ver, al inicio de casi cualquier hoja de estilos moderna, una regla como esta que aplica `border-box` a todo el documento de una sola vez:

```
* { box-sizing: border-box; }
```

CONCEPTO

display : cómo fluye cada elemento

La propiedad `display` define cómo se acomoda un elemento respecto a los demás. Cada etiqueta HTML trae un `display` por defecto: `<div>` , `<p>` , `<h1>` son `block` ; `<span>` , `<a>` , `<strong>` son `inline` .

display: block :

- Ocupa **todo el ancho disponible** del contenedor, aunque el contenido sea corto.
- Siempre empieza en una **línea nueva** — el elemento siguiente se acomoda debajo, no al lado.
- **Sí respeta** `width` , `height` , y todo el `padding/margin` del box model.

```
div { display: block; background: #16453f; }  
<div>Uno</div><div>Dos</div>  
<!-- "Uno" y "Dos" quedan en líneas separadas -->
```

CONCEPTO

inline e inline-block: las otras dos opciones

Valor	¿Nueva línea?	¿Respetar width/height?	¿Respetar margin/padding vertical?
inline	No — fluye junto al contenido vecino	No, se ignoran	No, se ignoran
inline-block	No — fluye junto al contenido vecino	Sí	Sí
block	Sí	Sí	Sí

`inline-block` es el punto intermedio: se comporta como `inline` para no forzar un salto de línea (útil para varios botones seguidos), pero como `block` para permitir controlar su tamaño exacto con `width / height` y su espaciado con `padding/margin`.

```
a.boton { display: inline-block; width: 120px; padding: 10px; }
<a class="boton">Uno</a><a class="boton">Dos</a>
<!-- Quedan lado a lado, cada uno con su tamaño respetado -->
```


INTERACCIÓN

Denle estilo a este HTML semántico

En parejas, 8 minutos: este fragmento ya está bien estructurado con etiquetas semánticas (de la sesión pasada). Escriban un CSS **mínimo** que use **selectores de clase** para darle: tipografía (`font-family` / `font-size`), un color de texto o fondo, y espaciado (padding y/o margin).

```
<article class="tarjeta">
  <h2 class="tarjeta-titulo">Anuncio</h2>
  <p class="tarjeta-texto">Clase suspendida el viernes.</p>
</article>
```

8 minutos · pista: empiecen por `.tarjeta { }`, luego afinen `.tarjeta-titulo` y `.tarjeta-texto` por separado

SÍNTESIS

Lo que hay que llevarse de hoy

1. CSS separa **contenido** (HTML) de **presentación**; en la práctica se vincula casi siempre como archivo **externo**, por mantenibilidad y cacheo del navegador.
2. Una regla CSS es `selector { propiedad: valor; }`; los selectores más comunes son de elemento, clase (`.clase`) e id (`#id`), y hay pseudo-clases como `:hover` .
3. Cuando dos reglas compiten por el mismo elemento, gana el más específico: **id > clase > elemento**.
4. Colores en nombre, hex o `rgb()` ; unidades absolutas (`px`) vs. relativas (`%` , `em` , `rem`).
5. El **modelo de caja** es `content` → `padding` → `border` → `margin`; por defecto el `width` solo cubre el `content`, y `box-sizing: border-box` hace que incluya también `padding` y `border`.
6. `display: block` , `inline` e `inline-block` determinan si un elemento salta de línea y si respeta tamaño y espaciado.



CIERRE

Antes de la próxima sesión

- Crear un archivo `.css` externo y vincularlo a una página HTML propia con `<link rel="stylesheet">` .
- Practicar el modelo de caja: cambiar `width` , `padding` , `border` y `margin` de un elemento y observar el efecto con y sin `box-sizing: border-box` .

Próxima sesión: Flexbox, Grid y diseño responsivo — cómo organizar varias cajas para construir layouts reales de páginas completas, y cómo adaptarlos a distintos dispositivos.

